

ATS API Input and Output Schemas

Day 16 – API Contract Documentation

1. Overview

The ATS API uses structured input and output schemas to ensure consistent communication between backend systems and the resume screening modules.

Request validation is implemented using Pydantic models.

The primary API schemas cover:

- Resume parsing
- Candidate scoring
- Candidate shortlisting
- Standard API responses
- Error responses
- Asynchronous job information

2. Standard Response Structure

Successful API operations use the following structure:

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}
```

Fields

Field	Type	Description
success	Boolean	Indicates successful operation
message	String	Human-readable result description
data	Object	Result generated by the operation

3. Standard Error Structure

Errors follow a structured format:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Description of the error."
  }
}
```

Fields

Field	Type	Description
success	Boolean	Always false for errors
error.code	String	Machine-readable error identifier
error.message	String	Human-readable error description

4. Resume Upload Input

Endpoint:

```
POST /api/v1/resumes/upload
```

The endpoint accepts a multipart file upload.

Supported resume formats:

```
.pdf
.docx
```

The resume file must contain data and cannot be empty.

5. Resume Upload Output

Example:

```
{
  "success": true,
  "message": "Resume uploaded successfully.",
  "data": {
    "resume_id": "unique-resume-id",
  }
}
```

```
      "job_id": "unique-job-id",
      "status": "QUEUED"
    }
  }
```

Output Fields

Field	Type	Description
resume_id	String	Unique identifier for uploaded resume
job_id	String	Processing job identifier
status	String	Current processing state

6. Resume Parsing Input Schema

Endpoint:

```
POST /api/v1/resumes/parse
```

Schema:

```
{
  "resume_text": "Candidate resume text"
}
```

Validation

resume_text:

- Required
- Must be a string
- Must contain at least one character

7. Resume Parsing Output Schema

Example:

```
{
  "success": true,
  "message": "Resume parsing completed.",
  "data": {
```

```
    "text_length": 68,
    "status": "parsed",
    "integration": "Existing resume parser pipeline can be connected here."
  }
}
```

The current Day 16 endpoint defines and validates the parsing contract and provides an integration point for the existing parser pipeline.

8. Candidate Scoring Input Schema

Endpoint:

```
POST /api/v1/scoring
```

Schema:

```
{
  "job_role": "Data Scientist",
  "scores": {
    "skill_match": 90,
    "experience_relevance": 80,
    "education_alignment": 75,
    "semantic_similarity": 85
  }
}
```

9. Candidate Score Fields

Field	Type	Range
skill_match	Number	0–100
experience_relevance	Number	0–100
education_alignment	Number	0–100
semantic_similarity	Number	0–100

All four evaluation scores are required.

10. Candidate Scoring Output

Tested response:

```
{
  "success": true,
  "message": "Candidate scoring completed.",
  "data": {
    "job_role": "Data Scientist",
    "overall_score": 85.0,
    "recommendation": "Strong Candidate",
    "score_breakdown": {
      "skill_match": 90.0,
      "experience_relevance": 80.0,
      "education_alignment": 75.0,
      "semantic_similarity": 85.0
    }
  }
}
```

Output Fields

Field	Description
job_role	Role used for ATS scoring
overall_score	Weighted ATS candidate score
recommendation	Candidate recommendation
score_breakdown	Individual evaluation scores

11. Shortlisting Input Schema

Endpoint:

```
POST /api/v1/shortlisting
```

Schema:

```
{
  "candidates": [
    {
      "candidate_id": "C001",
      "name": "Candidate A",
      "score": 91
    }
  ]
}
```

```
]
}
```

Each candidate contains:

Field	Type	Description
candidate_id	String	Unique candidate identifier
name	String	Candidate display name
score	Number	ATS score from 0–100

12. Shortlisting Output

The shortlisting endpoint returns the structured output generated by the existing candidate shortlisting module.

The output can contain:

- Total candidate count
- Shortlisted count
- Review count
- Rejected count
- Threshold configuration
- Ranked candidate list
- Candidate decisions
- Top candidate list

Example decision values:

```
Shortlisted
Review
Rejected
```

13. Job Status Schema

Endpoint:

```
GET /api/v1/jobs/{job_id}
```

Example:

```
{
  "success": true,
  "message": "Job status retrieved.",
  "data": {
    "job_id": "job-123",
    "job_type": "RESUME_PROCESSING",
    "status": "PROCESSING",
    "result": null,
    "error": null
  }
}
```

14. Job Status Values

Supported states:

```
QUEUED
PROCESSING
COMPLETED
FAILED
```

Meaning:

Status	Description
QUEUED	Job has been accepted
PROCESSING	Processing has started
COMPLETED	Processing finished successfully
FAILED	Processing encountered an error

15. Validation Error

Pydantic automatically rejects invalid API input.

For example:

```
{
  "skill_match": 150
}
```

is invalid because ATS scores must remain between:

```
0 and 100
```

The API returns:

```
HTTP 422
```

This validation was successfully tested during Day 16 implementation.

16. Resource Not Found Error

If an invalid job ID is requested:

```
GET /api/v1/jobs/non-existing-job
```

the API returns:

```
HTTP 404
```

with a structured error containing:

```
RESOURCE_NOT_FOUND
```

This behavior was also validated through automated testing.

17. Schema Benefits

Structured schemas provide:

- Consistent backend communication
- Input validation
- Predictable response formats
- Easier frontend integration
- Reduced malformed data
- Clear API documentation
- Easier automated testing

- Improved maintainability
-

18. Conclusion

The Day 16 input and output schemas establish standardized contracts for communication between backend applications and ATS AI modules.

Pydantic validation ensures that candidate information and scoring values satisfy expected constraints before entering the ATS pipeline, while standardized responses make the resulting data predictable and easier to integrate with future backend and recruiter-facing applications.